Lecture Notes: Ensemble Learning

CIC1205 - Machine Learning

Eduardo Bezerra

April 12, 2026

0 Contents

1 Introduction **2**
1.1 What Is Ensemble Learning . . . 2
1.2 How Ensembles Work . . . 2
1.3 Main Ensemble Paradigms . . . 3

2 Bagging **3**
2.1 Motivation and Overview . . . 3
2.2 Algorithmic Procedure . . . 4
2.3 Bootstrap Sampling . . . 4
2.4 Variance Reduction . . . 6
2.5 Worked Example . . . 7
2.6 Implementation in Scikit-learn . . . 8

3 Boosting **9**
3.1 Motivation and Overview . . . 9
3.2 Sequential Construction of the Ensemble . . . 9
3.3 Worked Example . . . 10
3.4 Implementation in Scikit-learn . . . 11
3.5 Why Boosting Reduces Bias . . . 12
3.6 The Learning Rate in Boosting . . . 15
3.7 Comparison: Bagging vs. Boosting . . . 15

4 Stacking **16**
4.1 Motivation and Overview . . . 16
4.2 Architecture . . . 17
4.3 Stacking Without Cross-Validation . . . 17
4.4 Stacking With Cross-Validation . . . 17
4.5 Worked Example . . . 18
4.6 Bias and Variance in Stacking . . . 19
4.7 Implementation in Scikit-learn . . . 22

5 Comparison of Ensemble Methods **23**
5.1 Bias and Variance Considerations . . . 23
5.2 When to Use Each Method . . . 24
5.3 Common Pitfalls . . . 24

6 Modern Extensions **24**
6.1 Random Forests . . . 24
6.2 Gradient Boosting Machines . . . 25
6.3 Deep Ensembles . . . 25
6.4 Nested Ensembles . . . 26

7 Final Remarks **28**
7.1 Choosing the Right Ensemble Method . . . 28
7.2 Validation of Ensemble Methods . . . 28

1 Introduction

1.1 What Is Ensemble Learning

Ensemble learning is a methodology in which multiple supervised learning models are combined to form a single predictive system. The resulting model is called an **ensemble**, and the individual models that compose it are called **base models** or **base learners**.

Each base model may have different characteristics, biases, or random initialization. When their predictions are combined, the ensemble aims to achieve higher accuracy and robustness than any of its components individually.

Ensemble

An **ensemble** is a predictive model obtained by combining the outputs of multiple base learners. The combination aims to produce better generalization performance than any individual component, by leveraging the diversity of errors made by different models.

Key idea. The intuition behind ensemble learning is that combining diverse opinions often leads to better decisions. If each base learner makes independent errors, these errors can cancel out when aggregated, improving overall performance.

Analogy. Consider asking a single expert for their opinion on a complex question. That expert may be knowledgeable but is still subject to blind spots and biases. Now consider asking a diverse panel of experts and averaging their answers. Even if each individual is imperfect, the collective judgment tends to be more reliable.

Scope. Ensemble methods are applicable to both classification and regression tasks, and sometimes to ranking or anomaly detection. They can be used with a wide range of base models, from decision trees to neural networks.

1.2 How Ensembles Work

At inference time, an ensemble combines the predictions of all base models to produce the final output. This combination can occur in various ways, depending on the task:

- For classification, the final label is usually determined by **majority voting**.
- For regression, the final output is typically the **average** of the base predictions.

Mathematically, let $h_t(\mathbf{x})$ be the prediction of the t-th base model for input $\mathbf{x}$. Then the ensemble prediction $H(\mathbf{x})$ is given by

$$H(\mathbf{x}) = \begin{cases} \text{mode}\{h_1(\mathbf{x}), \ldots, h_T(\mathbf{x})\}, & \text{for classification,} \\ \dfrac{1}{T} \sum_{t=1}^{T} h_t(\mathbf{x}), & \text{for regression.} \end{cases}$$

Goal. The ensemble should reduce both the variance and the bias of the prediction, depending on the diversity and nature of its base learners.

1.3 Main Ensemble Paradigms

Ensemble methods are not defined by a single technique, but rather by a family of strategies that differ in how they generate and combine multiple models. Despite their differences, all ensemble approaches rely on a central principle: combining diverse predictors can lead to improved generalization performance.

Three dominant paradigms have emerged in the literature, each characterized by a distinct mechanism for promoting diversity and reducing prediction error:

Paradigm	**Primary Goal**	**Construction**
Bagging	Reduce variance	Parallel, with bootstrap resampling
Boosting	Reduce bias	Sequential, with error reweighting
Stacking	Combine strengths	Meta-model learned from base outputs

Bagging constructs multiple versions of the same base model by training them on different bootstrap samples of the data, and aggregates their predictions to stabilize the output and reduce variance. In contrast, boosting builds models sequentially, where each new learner focuses on correcting the mistakes of its predecessors, thereby reducing bias and improving accuracy. Finally, stacking adopts a more flexible strategy by training a secondary model to learn how to optimally combine the predictions of a set of (possibly heterogeneous) base learners.

These paradigms illustrate that there is no single path to building an effective ensemble: diversity can be induced through data perturbation, sequential refinement, or model heterogeneity. In the following sections, we examine each of these approaches in detail, highlighting their underlying assumptions, strengths, and limitations.

Entry Ticket 1

In your own words, explain why combining multiple imperfect models can yield better performance than any single model. What conditions must hold for this improvement to materialize?

2 Bagging

2.1 Motivation and Overview

Bagging (short for *bootstrap aggregation*) is an ensemble method designed to improve the stability and accuracy of machine learning algorithms. It is particularly effective for models that are **unstable**: models for which small changes in the training data lead to large changes in predictions.

Bagging

Bagging is an ensemble strategy that trains T base models independently, each on a different bootstrap sample of the training data, and aggregates their predictions by majority vote (classification) or averaging (regression).

Main idea. Instead of training a single model on the available data, Bagging trains several models on different **bootstrap samples** of the dataset and combines their predictions. Each base model learns slightly different patterns due to differences in its training subset, and the aggregation of these independent models reduces the overall prediction variance.

When to use. Bagging is especially beneficial when using high-variance base learners, such as decision trees or neural networks. For algorithms that are already stable (e.g., k-nearest neighbors with large k), Bagging provides little to no improvement.

2.2 Algorithmic Procedure

Given a training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, Bagging proceeds as follows:

1. For $t = 1, 2, \ldots, T$:
   (a) Generate a bootstrap sample $\mathcal{D}_t$ of size m by sampling examples from $\mathcal{D}$ *with replacement*.
   (b) Train a base learner h_t using $\mathcal{D}_t$.
2. After training, combine the predictions of the T models through aggregation.

Each base learner h_t is trained independently of the others. The randomness introduced by bootstrap sampling promotes diversity among them, which is the key mechanism for variance reduction.

Final prediction. At inference time, the ensemble prediction for a new input $\mathbf{x}$ is

$$H(\mathbf{x}) = \begin{cases} \text{mode}\{h_1(\mathbf{x}), \ldots, h_T(\mathbf{x})\}, & \text{classification} \\ \dfrac{1}{T}\sum_{t=1}^T h_t(\mathbf{x}), & \text{regression.} \end{cases}$$

2.3 Bootstrap Sampling

The bootstrap principle comes from statistics and refers to resampling *with replacement* from an original dataset. Each bootstrap sample contains approximately 63.2% of the *unique* observations from the original data, with some examples appearing multiple times and others not at all.

Derivation. The probability that a given example is *not* selected in any single draw is $1 - \frac{1}{m}$. The probability it is never selected across m independent draws is

$$\Pr(\text{absent}) = \left(1 - \frac{1}{m}\right)^m \xrightarrow{m\to\infty} e^{-1} \approx 0.368.$$

Therefore, each bootstrap sample includes approximately $1 - e^{-1} \approx 63.2\%$ of the original unique examples.

Numerical convergence. Table 1 shows how quickly the expression $(1 - 1/m)^m$ converges to e^{-1} as the dataset grows. Even for small datasets ($m \approx 20$) the approximation is already very close to the theoretical limit.

Table 1: Probability that a given example is *absent* from a bootstrap sample of size m, compared with the theoretical limit e^{-1}.

m	$(1 - 1/m)^m$	Unique examples kept ($\approx$)
5	0.3277	67.2%
10	0.3487	65.1%
20	0.3585	64.2%
50	0.3642	63.6%
100	0.3660	63.4%
1000	0.3677	63.2%
∞	$e^{-1} \approx 0.3679$	$1 - e^{-1} \approx 63.2\%$

Interpretation. Each sample represents a *perturbation* of the original dataset. Training different models on these perturbations induces variability in the learned hypotheses, which is essential for variance reduction when aggregating predictions. Figure 1 illustrates the three possible roles an example can play in a given bootstrap sample.

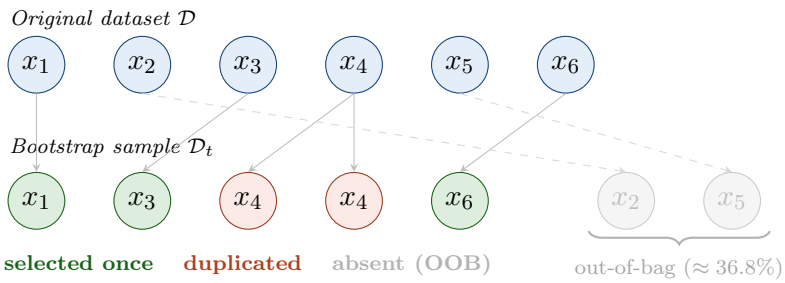

Figure 1: One bootstrap sample drawn from a dataset of $m = 6$ examples. Green nodes were selected once; orange nodes were drawn more than once (here x_4 appears twice); gray nodes were not selected and form the **out-of-bag (OOB)** set, available as a free validation set. On average, about 63.2% of unique examples appear in each sample.

Out-of-bag (OOB) samples. The examples left out of a bootstrap sample (approximately 36.8% of the data) are called **out-of-bag (OOB) samples**. Because each base model never sees its OOB examples during training, these can be used to estimate generalisation error *without* a separate holdout set. Concretely, for each training example $(\mathbf{x}_i, y_i)$, only the models for which $(\mathbf{x}_i, y_i)$ was OOB are used to predict y_i; the average of those predictions constitutes the OOB estimate.

Interactive visualisation

A standalone interactive simulator (HTML/JavaScript) accompanies these notes. It lets you choose the dataset size m, draw individual bootstrap samples to inspect

which examples are selected, duplicated, or left out, and accumulate hundreds of draws to observe the empirical convergence of the unique-example fraction towards the theoretical value $1 - e^{-1} \approx 63.2\%$.

2.4 Variance Reduction

The variance of an ensemble prediction can be shown to be lower than that of the individual base models, provided their errors are not perfectly correlated.

Let $\hat{y}_t$ denote the prediction of model t and $\bar{y} = \frac{1}{T}\sum_t \hat{y}_t$ the ensemble prediction. If each model has variance σ^2 and pairwise correlation ρ, then

$$\mathrm{Var}(\bar{y}) = \rho\sigma^2 + \frac{1-\rho}{T}\sigma^2.$$

As T increases, the second term decreases, leading to reduced overall variance. The first term $\rho\sigma^2$ represents an irreducible floor: if all models make perfectly correlated errors ($\rho = 1$), ensemble averaging provides no benefit. This highlights the importance of **diversity** among base learners.

Drafted Proof. Suppose an ensemble prediction is formed by averaging T base-model predictions:

$$\hat{f}_{\text{ens}}(x) = \frac{1}{T}\sum_{m=1}^{T} \hat{f}_m(x).$$

Assume that each base prediction has the same variance,

$$\mathrm{Var}(\hat{f}_m(x)) = \sigma^2,$$

and that the pairwise correlation between any two distinct base predictions is ρ, so that

$$\mathrm{Cov}(\hat{f}_i(x), \hat{f}_j(x)) = \rho\sigma^2 \qquad (i \neq j).$$

Then

$$\mathrm{Var}\Big(\hat{f}_{\text{ens}}(x)\Big) = \mathrm{Var}\left(\frac{1}{T}\sum_{m=1}^{T} \hat{f}_m(x)\right) = \frac{1}{T^2}\left(\sum_{m=1}^{T} \mathrm{Var}(\hat{f}_m(x)) + \sum_{i\neq j} \mathrm{Cov}(\hat{f}_i(x), \hat{f}_j(x))\right).$$

Substituting the common-variance and common-correlation assumptions gives

$$\mathrm{Var}\Big(\hat{f}_{\text{ens}}(x)\Big) = \frac{1}{T^2}\left(M\sigma^2 + T(T-1)\rho\sigma^2\right) = \sigma^2\left(\rho + \frac{1-\rho}{T}\right).$$

This expression makes the benefit of ensembling explicit. If the base models are not perfectly correlated ($\rho < 1$), then averaging reduces variance. The reduction is strongest when correlations are small, which is why ensemble methods benefit not only from accurate base learners but also from diverse ones [Hastie et al., 2009, Breiman, 1996].

Variance–Diversity Trade-off

Bagging reduces variance by aggregating diverse predictions. The benefit grows as model correlation ρ decreases and as the number of base models T increases. Bagging does *not* systematically reduce bias.

2.5 Worked Example

To make the mechanism of Bagging concrete, consider a toy dataset $\mathcal{D}$ with three training samples, two input features, and a binary target:

$$\mathcal{D} = \begin{bmatrix} 1 & 2 & | & 0 \\ 0 & 1 & | & 1 \\ 2 & 0 & | & 0 \end{bmatrix}$$

We train $T = 3$ base models, each on a different bootstrap sample.

Step 1: Generating Bootstrap Samples

Each bootstrap sample is created by randomly drawing (with replacement) three rows from $\mathcal{D}$:

$$\begin{aligned} \mathcal{D}_1 &= \{(1,2,0),\ (2,0,0),\ (1,2,0)\} \\ \mathcal{D}_2 &= \{(0,1,1),\ (2,0,0),\ (0,1,1)\} \\ \mathcal{D}_3 &= \{(2,0,0),\ (0,1,1),\ (2,0,0)\} \end{aligned}$$

Each sample contains repetitions, illustrating the effect of resampling with replacement.

Step 2: Training the Base Models

We train three base learners (e.g., decision trees) on these datasets:

$$h_1 = \text{DecisionTree}(\mathcal{D}_1), \quad h_2 = \text{DecisionTree}(\mathcal{D}_2), \quad h_3 = \text{DecisionTree}(\mathcal{D}_3).$$

Step 3: Making Predictions

When applied to the original dataset $\mathcal{D}$, each model produces a predicted label per row:

$$h_1(\mathcal{D}) \to [0,\ 0,\ 0], \quad h_2(\mathcal{D}) \to [1,\ 1,\ 0], \quad h_3(\mathcal{D}) \to [0,\ 1,\ 0].$$

Step 4: Aggregating Predictions

For classification, Bagging combines predictions by majority vote. The ensemble output $H(\mathcal{D})$ is:

$$H(\mathcal{D}) = \begin{bmatrix} \text{mode}(0,1,0) & | & 0 \\ \text{mode}(0,1,1) & | & 1 \\ \text{mode}(0,0,0) & | & 0 \end{bmatrix}$$

yielding final predictions [0, 1, 0]. Even when individual models disagree, their aggregated decision tends to be more stable and accurate.

Figure 2 illustrates the bagging (bootstrap aggregating) procedure. Starting from the original dataset $\mathcal{D}$, multiple bootstrap samples $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3$ are generated by sampling with replacement. Each of these datasets is then used to train an independent base model (h_1, h_2, h_3), typically of the same type but exposed to slightly different data distributions. The predictions of these models are subsequently combined through an aggregation mechanism (majority voting in classification tasks or averaging in regression) yielding the final ensemble prediction $H(\mathbf{x})$. This process promotes diversity among the base learners, which is essential for reducing variance and improving generalization performance.

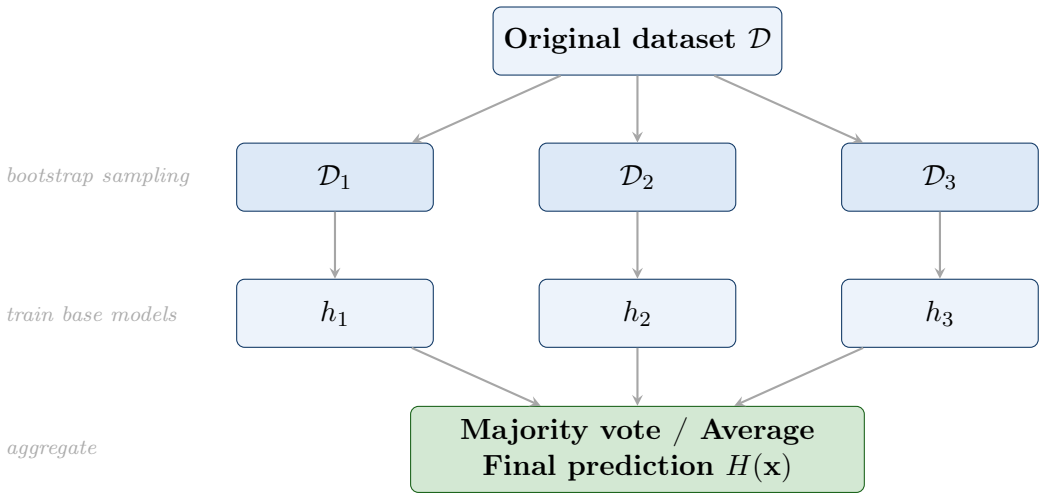

Figure 2: Bagging: multiple bootstrap samples are drawn from the original dataset, each used to train an independent base model. Their predictions are aggregated (by majority vote for classification or by averaging for regression) to form the final ensemble output $H(\mathbf{x})$.

Entry Ticket 2

Suppose you train a Bagging ensemble with $T = 100$ decision trees on a dataset of 1 000 examples.

1. How many unique training examples does each bootstrap sample contain, approximately?
2. What happens to the variance of the ensemble as $T \to \infty$? Does the bias change?
3. Why would Bagging provide little benefit when applied to logistic regression?

2.6 Implementation in Scikit-learn

Listing 1 presents a practical implementation of the bagging paradigm using scikit-learn. In this example, multiple decision trees are trained independently on different bootstrap samples of the training data, as controlled by the `bootstrap=True` parameter. The number of such models is specified by `n_estimators`, with each tree learning from a slightly perturbed version of the original dataset. Because these models are trained in parallel and without interaction, their errors tend to be less correlated. The final prediction is ob-

tained by aggregating the outputs of all base learners (typically through majority voting in classification tasks) resulting in a more stable and lower-variance model.

```
...

bag_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=10,   # number of bootstrap models
    bootstrap=True,
    random_state=42
)

bag_model.fit(X_train, y_train)
y_pred = bag_model.predict(X_test)
print(f"Bagging accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

Listing 1: Bagging classifier with decision trees in scikit-learn

3 Boosting

3.1 Motivation and Overview

Boosting is another major family of ensemble methods. Unlike Bagging, which trains independent models in parallel, Boosting builds models in a **sequential** manner: each new model attempts to correct the errors made by the previous ones.

Boosting

Boosting is an ensemble strategy that constructs a strong learner by sequentially training base models, each focused on the mistakes of its predecessors. Predictions are combined through a weighted vote, where more accurate models receive higher weight.

Key idea. Boosting combines several **weak learners** (models that perform only slightly better than random guessing) into a strong learner with high predictive accuracy. This is achieved by assigning higher weight to training examples that were misclassified or poorly predicted by previous models.

When to use. Boosting is particularly suitable for base learners that have high bias and low variance, such as shallow decision trees (often called *decision stumps*). By iteratively focusing on difficult examples, Boosting reduces bias, which is the dominant source of error for weak learners.

3.2 Sequential Construction of the Ensemble

The general workflow of Boosting can be summarized as follows:

1. Initialize the model with equal weights for all training examples.
2. Train the first weak learner on the weighted dataset.

3. Increase the weights of examples that were predicted incorrectly.
4. Train the next weak learner using the updated weights.
5. Combine all weak learner predictions through a weighted vote.

Each new base model is therefore *dependent* on the performance of those built before it. Through this iterative correction mechanism, the ensemble concentrates progressively on the hardest cases, leading to a model that performs significantly better than its individual parts.

3.3 Worked Example

To illustrate how Boosting incrementally improves model performance, consider the same toy dataset $\mathcal{D}$ with three samples:

$$\mathcal{D} = \begin{bmatrix} 1 & 2 & | & 0 \\ 0 & 1 & | & 1 \\ 2 & 0 & | & 0 \end{bmatrix}$$

We construct a Boosting ensemble using $T = 3$ weak learners (decision stumps: depth-1 trees).

Step 1: Initialize Uniform Weights

All samples initially receive equal weights:

$$w_1 = w_2 = w_3 = \tfrac{1}{3}.$$

Step 2: Train the First Weak Learner

A decision stump h_1 is trained on the current weights. Suppose it predicts:

$$h_1(\mathcal{D}) = [0,\ 0,\ 0].$$

Only the second example is misclassified (true label = 1). The weighted error is:

$$\varepsilon_1 = \tfrac{1}{3}.$$

The model influence (its weight in the final ensemble) is:

$$\alpha_1 = \tfrac{1}{2} \ln\left(\frac{1-\varepsilon_1}{\varepsilon_1}\right) = \tfrac{1}{2}\ln(2) \approx 0.35.$$

Step 3: Update Sample Weights

Misclassified examples receive higher weights:

$$w_i^{(2)} = w_i^{(1)} \cdot e^{\alpha_1 \cdot \mathbf{1}[y_i \neq h_1(\mathbf{x}_i)]}.$$

After normalization: $\mathbf{w}^{(2)} = [0.25,\ 0.50,\ 0.25]$.

Step 4: Train the Second Weak Learner

The new model h_2 is trained on the reweighted data and predicts:

$$h_2(\mathcal{D}) = [0,\ 1,\ 1].$$

Now the third example is misclassified:

$$\varepsilon_2 = 0.25, \qquad \alpha_2 = \tfrac{1}{2}\ln(3) \approx 0.55.$$

Weights are updated again: $\mathbf{w}^{(3)} = [0.22,\ 0.33,\ 0.45]$.

Step 5: Train the Third Weak Learner

The final weak learner h_3 is trained on the updated weights and predicts:

$$h_3(\mathcal{D}) = [0,\ 1,\ 0].$$

Weighted error $\varepsilon_3 = 0.15$, and corresponding influence $\alpha_3 \approx 0.87$.

Step 6: Combine the Models

The final Boosting classifier is the weighted combination of the three weak learners:

$$H(\mathbf{x}) = \operatorname{sign}(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}) + \alpha_3 h_3(\mathbf{x}))\,.$$

Figure 3 illustrates the sequential construction of this ensemble. The process begins with the original dataset $\mathcal{D}$, where all training examples are initially assigned uniform weights, indicating equal importance. A first weak learner h_1 is trained on this dataset, producing an initial set of predictions. Based on its errors, the weights of the training examples are then updated: misclassified instances receive higher weights, while correctly classified ones are downweighted. This reweighted dataset is used to train the next learner h_2, which is thus encouraged to focus on the mistakes of h_1. The procedure is repeated iteratively, with each subsequent learner (e.g., h_3) concentrating increasingly on the hardest examples in the dataset. Finally, the individual models are combined through a weighted aggregation, where each learner contributes according to its performance, yielding the final ensemble prediction $H(\mathbf{x}) = \sum_t \alpha_t h_t(\mathbf{x})$. This sequential dependency between learners is the defining characteristic of boosting and is what enables it to progressively reduce bias and improve predictive accuracy.

3.4 Implementation in Scikit-learn

Listing 2 illustrates a practical implementation of the boosting paradigm using the AdaBoost algorithm in scikit-learn. In this example, the base learners are decision stumps (i.e., shallow decision trees with depth one), which are sequentially trained to correct the mistakes of previous models. The parameter `n_estimators` controls the number of boosting rounds, while `learning_rate` determines the contribution of each learner to the final ensemble. As training progresses, the algorithm places increasing emphasis on misclassified examples, effectively focusing on the hardest cases in the dataset. The final prediction is obtained through a weighted combination of the individual learners, reflecting their relative performance.

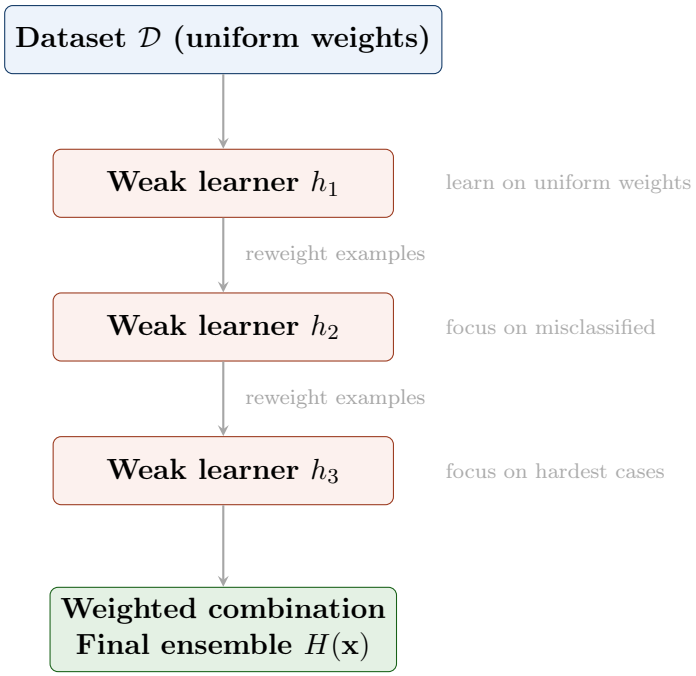

Figure 3: Boosting: each weak learner is trained sequentially. After each round, example weights are updated to emphasize the mistakes of the current ensemble. The final model is a weighted combination $\sum_t \alpha_t h_t(\mathbf{x})$, where α_t reflects the accuracy of each component.

```
...

boost_model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

boost_model.fit(X_train, y_train)
y_pred = boost_model.predict(X_test)
print(f"Boosting accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

Listing 2: AdaBoost classifier with decision stumps in scikit-learn

3.5 Why Boosting Reduces Bias

Intuition

A *weak learner* is a model whose predictions are only marginally better than random guessing; its error arises primarily from **bias**: the model is too simple to capture the true decision boundary. Averaging several such models (as Bagging does) cannot fix this: the average of many slightly-wrong predictions is still slightly wrong.

Boosting takes a different approach. After training the first weak learner h_1, the *residual error* (the systematic pattern that h_1 failed to capture) is made explicitly visible by

increasing the weights of the misclassified examples. The next learner h_2 is then trained on this re-weighted distribution, so it is forced to focus precisely on the structure that h_1 missed. Each round peels away one layer of the remaining systematic error, progressively reducing bias.

Crucially, this works because the errors are **systematic** (bias), not random (variance). A biased model fails on the same examples consistently across different training sets; reweighting those examples makes the failure mode explicit and correctable.

Formal argument: exponential error decay in AdaBoost

We now show that, under a mild condition on the weak learners, the training error of AdaBoost decreases *exponentially* with the number of rounds T. This is the direct analogue of the variance-reduction proof for Bagging (Section 2.4).

Setup. Consider a binary classification task with labels $y_i \in \{-1, +1\}$. Let ε_t be the weighted error of the t-th weak learner h_t, and let $\alpha_t = \frac{1}{2}\ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$ be its influence weight. The AdaBoost ensemble after T rounds is

$$H(\mathbf{x}) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t\, h_t(\mathbf{x})\right), \qquad F_T(\mathbf{x}) := \sum_{t=1}^{T} \alpha_t\, h_t(\mathbf{x}).$$

Training error bound. The training error of H satisfies

$$\frac{1}{m}\sum_{i=1}^{m} \mathbf{1}\big[H(\mathbf{x}_i) \neq y_i\big] \;\leq\; \frac{1}{m}\sum_{i=1}^{m} e^{-y_i F_T(\mathbf{x}_i)}.$$

This follows from the inequality $\mathbf{1}[z \leq 0] \leq e^{-z}$ for all $z \in \mathbb{R}$.

Recursive decomposition. Let $w_i^{(1)} = 1/m$ be the initial uniform weights. After each round t, AdaBoost updates the weights as

$$w_i^{(t+1)} = \frac{w_i^{(t)}\, e^{-\alpha_t y_i h_t(\mathbf{x}_i)}}{Z_t},$$

where Z_t is the normalisation constant. One can verify that

$$Z_t = 2\sqrt{\varepsilon_t(1-\varepsilon_t)}.$$

Unrolling the recursion from $t = 1$ to T gives

$$\frac{1}{m}\sum_{i=1}^{m} e^{-y_i F_T(\mathbf{x}_i)} = \prod_{t=1}^{T} Z_t = \prod_{t=1}^{T} 2\sqrt{\varepsilon_t(1-\varepsilon_t)}.$$

Weak learner condition. Suppose each weak learner satisfies $\varepsilon_t \leq \frac{1}{2} - \gamma$ for some fixed $\gamma > 0$ (i.e. each learner does better than random guessing by at least a margin γ). Then

$$2\sqrt{\varepsilon_t(1-\varepsilon_t)} \;\leq\; 2\sqrt{\left(\tfrac{1}{2}-\gamma\right)\left(\tfrac{1}{2}+\gamma\right)} = \sqrt{1-4\gamma^2} \;\leq\; e^{-2\gamma^2},$$

where the last step uses $\sqrt{1-u} \leq e^{-u/2}$ for $u \in [0, 1]$.

Exponential decay. Combining the above:

$$\boxed{\frac{1}{m}\sum_{i=1}^{m}\mathbf{1}\big[H(\mathbf{x}_i)\neq y_i\big] \;\leq\; \prod_{t=1}^{T}2\sqrt{\varepsilon_t(1-\varepsilon_t)} \;\leq\; e^{-2\gamma^2 T}.}$$

The training error decreases *exponentially* in T, as long as each weak learner performs at least marginally better than chance. This is the formal mechanism behind bias reduction: iterative reweighting guarantees that the ensemble's systematic error vanishes at an exponential rate.

Bias Reduction in AdaBoost

If each weak learner achieves weighted error $\varepsilon_t \leq \frac{1}{2}-\gamma$ for some $\gamma > 0$, the training error of the AdaBoost ensemble is bounded by $e^{-2\gamma^2 T}$. Adding more rounds always reduces training error, provided the weak learner condition holds. The parameter γ measures how much better than random guessing each learner is; a larger γ yields faster convergence.

Connecting bias and variance

The exponential decay result concerns *training* error, not generalisation error. In practice, running Boosting for too many rounds eventually overfits: the variance of the ensemble begins to increase as the model memorises noise in the training data, eventually dominating the gains from bias reduction.

This is the core trade-off for Boosting, symmetric to the one for Bagging:

	Bagging	**Boosting**
Primary effect	Variance $\downarrow$	Bias $\downarrow$
Secondary risk	Bias unchanged	Variance $\uparrow$ (overfitting)
Formal guarantee	$\mathrm{Var}(\bar{y}) = \rho\sigma^2 + \frac{1-\rho}{T}\sigma^2$	Training error $\leq e^{-2\gamma^2 T}$
Key condition	Low correlation ρ among learners	Weak learner margin $\gamma > 0$

Weak learner condition in practice

The condition $\varepsilon_t < \frac{1}{2}$ is much weaker than it may appear: it only requires that each model does *slightly* better than a coin flip on the weighted training distribution. Decision stumps (depth-1 trees) routinely satisfy this, which is why they are the canonical base learner for AdaBoost. If no weak learner with $\varepsilon_t < \frac{1}{2}$ can be found, AdaBoost assigns $\alpha_t \leq 0$ and the round effectively contributes nothing; the ensemble stops improving.

Entry Ticket 3

Consider an AdaBoost ensemble trained for $T = 50$ rounds, where each weak learner achieves a weighted error of $\varepsilon_t = 0.4$ (so $\gamma = 0.1$).

1. Using the bound $e^{-2\gamma^2 T}$, compute the theoretical upper bound on the training

error after 50 rounds.

2. What would happen to this bound if you doubled T to 100? And if you found stronger weak learners with $\varepsilon_t = 0.3$ ($\gamma = 0.2$)?
3. The bound guarantees low *training* error. Why does this not automatically guarantee low *test* error? What phenomenon can arise, and how is it typically mitigated?

3.6 The Learning Rate in Boosting

In practical implementations such as `AdaBoostClassifier`, a parameter called the *learning rate* η controls the contribution of each weak learner to the final ensemble. After computing the influence weight α_t, the learning rate scales it before the model is added:

$$H(\mathbf{x}) = \operatorname{sign}\left(\sum_{t=1}^{T} \eta\, \alpha_t\, h_t(\mathbf{x})\right).$$

Typical values are $\eta \in [0.01, 1.0]$. A smaller learning rate requires more boosting rounds to reach a given accuracy level, but often produces smoother and more generalizable models. This is because each individual update is less aggressive, reducing the risk of overfitting on particular examples.

Learning rate and number of estimators

There is a well-known trade-off between η and the number of base learners T:

- Use a smaller η (e.g., 0.01–0.1) together with a large T for more stable, regularized ensembles.
- Use a larger η (e.g., 0.5–1.0) with a smaller T for faster training.

The learning rate in Boosting plays a role analogous to the step size in gradient descent: it regulates how strongly each new model corrects the errors of the current ensemble.

3.7 Comparison: Bagging vs. Boosting

Although Bagging and Boosting are both ensemble methods, they pursue improvement through fundamentally different strategies.

Aspect	Bagging	Boosting
Learning style	Parallel (independent models)	Sequential (dependent models)
Best base model	High-variance learners	High-bias (weak) learners
Primary goal	Reduce variance	Reduce bias
Sampling strategy	Bootstrap resampling (uniform)	Weighted sampling (emphasis on errors)
Model combination	Simple averaging or majority vote	Weighted average (α_t-based)
Typical example	Random Forest	AdaBoost, Gradient Boosting

Conceptual difference. Bagging reduces overfitting by smoothing out noise across multiple independent predictions. Boosting combats underfitting by sequentially increasing model complexity toward the hardest examples.

Entry Ticket 4

A practitioner observes that her decision tree achieves 72% accuracy on a classification task but exhibits high variance across different data splits.

1. Which ensemble method (Bagging or Boosting) would you recommend? Justify your choice.
2. Would your answer change if the decision tree consistently achieved only 55% accuracy regardless of the data split? Why?
3. What would happen if Boosting used too many iterations on a noisy dataset?

4 Stacking

4.1 Motivation and Overview

Stacking (or *stacked generalization*) is an ensemble method that combines the predictions of multiple base models through a higher-level model, called a **meta-model** or **meta-learner**. Instead of using simple averaging or majority voting (as in Bagging and Boosting), Stacking *learns* how to combine the base models optimally.

Stacking

Stacking is an ensemble strategy with two layers: a first layer of diverse base models whose predictions become the input features for a second-layer meta-model, which learns how to optimally combine them to produce the final prediction.

Key idea. Different base models may capture different aspects of the data: one may handle non-linearities well, while another is more robust to noise. Stacking uses the predictions of several base learners as input features for a new learning algorithm that

learns how to blend their outputs, producing a final model that ideally captures the strengths of each component.

4.2 Architecture

The ensemble has two conceptual layers:

- **Level 1 (base models):** several models $h_t(\mathbf{x})$ are trained on the original training dataset.
- **Level 2 (meta-model):** a model $h'(\cdot)$ is trained to combine the predictions of the base models.

At inference time, the stacked ensemble produces the final prediction as:

$$H(\mathbf{x}) = h'\big(h_1(\mathbf{x}),\, h_2(\mathbf{x}),\, \ldots,\, h_T(\mathbf{x})\big).$$

Analogy. Stacking works like a committee where each member (base model) provides an opinion, and a chairperson (meta-model) learns how to best interpret and combine those opinions.

4.3 Stacking Without Cross-Validation

In the simplest form of Stacking, all base models are trained on the entire training dataset, and their predictions on that same dataset are used to train the meta-model.

Limitation. This approach risks overfitting: the base models are evaluated on the same data they were trained on. Their predictions tend to be overly optimistic, and the meta-model may learn to exploit artifacts specific to the training data that will not generalize to unseen examples.

4.4 Stacking With Cross-Validation

To prevent this overfitting, Stacking is typically implemented using cross-validation to produce **out-of-sample** predictions for the meta-model.

Procedure.

1. Split the dataset $\mathcal{D}$ into K folds.
2. For each fold k:
   (a) Train all base models h_{kt} on $\mathcal{D} \setminus \mathcal{D}_k$.
   (b) Obtain predictions on the held-out fold $\mathcal{D}_k$, producing $\mathcal{D}'_k$.
3. Combine all folds: $\mathcal{D}' = \bigcup_{k=1}^{K} \mathcal{D}'_k$.
4. Train the meta-model h' on $\mathcal{D}'$.
5. Retrain each base model h_t on the entire dataset $\mathcal{D}$.
6. The final prediction is $H(\mathbf{x}) = h'\big(h_1(\mathbf{x}), \ldots, h_T(\mathbf{x})\big)$.

Advantage. Because each example in $\mathcal{D}'$ is predicted by a base model that never saw it during training, the meta-model receives unbiased training data, leading to better generalization.

4.5 Worked Example

Consider a toy dataset $\mathcal{D}$ with three examples and three input features:

$$\mathcal{D} = \begin{bmatrix} 1 & 2 & 2 & | & 0 \\ 0 & 1 & 1 & | & 0 \\ 2 & 0 & 3 & | & 1 \end{bmatrix}$$

We use $T = 4$ base models: `LogisticRegression`, `NeuralNetwork`, `DecisionTreeClassifier`, and `KNeighborsClassifier`.

Step 1: Training the Base Models

Each model h_t is trained on $\mathcal{D}$:

$$h_{\text{LR}}, \quad h_{\text{NN}}, \quad h_{\text{DT}}, \quad h_{\text{KNN}}.$$

Step 2: Generating First-Level Predictions

Each trained model is applied to $\mathcal{D}$:

$$\begin{array}{lcl} h_{\text{LR}}(\mathcal{D}) & \to & [0,\ 0,\ 1] \\ h_{\text{NN}}(\mathcal{D}) & \to & [1,\ 1,\ 1] \\ h_{\text{DT}}(\mathcal{D}) & \to & [0,\ 0,\ 0] \\ h_{\text{KNN}}(\mathcal{D}) & \to & [1,\ 1,\ 0] \end{array}$$

Step 3: Building the Meta-Level Dataset $\mathcal{D}'$

These predictions form the features of a new dataset, with the original targets retained:

$$\mathcal{D}' = \begin{bmatrix} 0 & 1 & 0 & 1 & | & 0 \\ 0 & 1 & 0 & 1 & | & 0 \\ 1 & 1 & 0 & 0 & | & 1 \end{bmatrix}$$

Step 4: Training the Meta-Model

A meta-learner (e.g., logistic regression) is trained on $\mathcal{D}'$ to learn how to optimally combine the base predictions:

$$h' = \text{LogisticRegression}(\mathcal{D}').$$

Figure 4 illustrates the full stacking workflow for this example. The process begins with the original dataset $\mathcal{D}$, which is used to train multiple heterogeneous base models, such as logistic regression (h_{LR}), a neural network (h_{NN}), a decision tree (h_{DT}), and a k-nearest neighbors classifier (h_{KNN}). Each of these models produces its own predictions for the input instances, capturing different patterns and inductive biases present in the data.

These predictions are then collected and combined with the true target values to form a new dataset, denoted $\mathcal{D}'$, often referred to as the meta-dataset. In this representation, the outputs of the base models serve as input features for a second-level learner. A meta-model h' is subsequently trained on $\mathcal{D}'$ to learn how to best combine the base predictions, potentially correcting their individual weaknesses and leveraging their complementary strengths. The final prediction $H(\mathbf{x})$ is thus obtained through this learned combination, making stacking a flexible and powerful approach for integrating diverse models into a single predictive system.

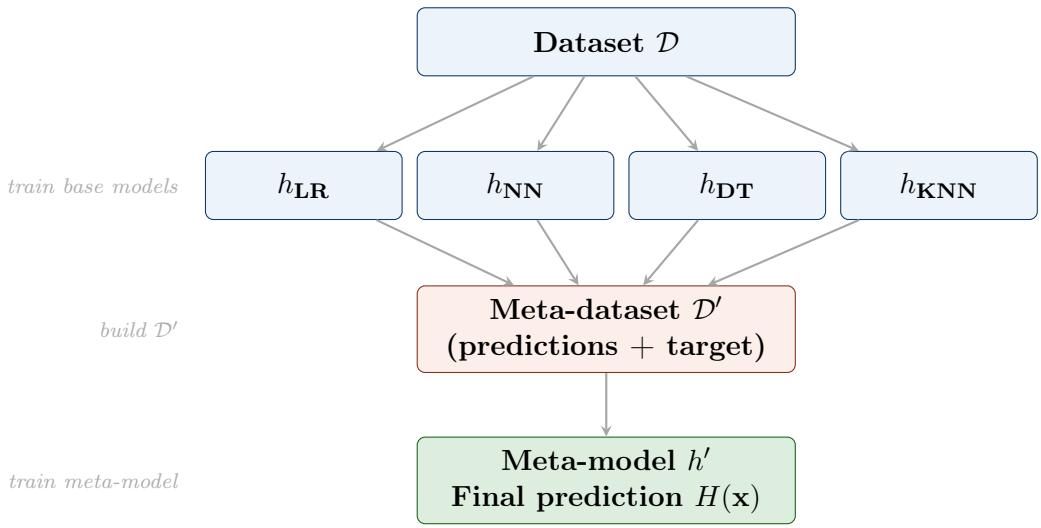

Figure 4: Stacking: each base model is trained on the original dataset $\mathcal{D}$ and its predictions are used as features of the meta-dataset $\mathcal{D}'$. The meta-model h' is then trained on $\mathcal{D}'$ to learn the optimal combination of base predictions.

4.6 Bias and Variance in Stacking

Unlike Bagging and Boosting, Stacking does not have a single clean theoretical guarantee analogous to the variance-reduction formula of Section 2.4 or the exponential error-decay bound of Section 3.5. This is not a gap in the analysis; it reflects a genuine property of the method: the bias–variance behaviour of a Stacking ensemble depends on *design choices* rather than on the aggregation mechanism alone. Understanding why requires examining the two levels of the architecture separately.

Effect on variance

Each Level-1 base model is trained on the *same* original dataset $\mathcal{D}$. This is the key structural difference from Bagging, where each model sees a different bootstrap sample. Because the base models share the same training data, their predictions tend to be more correlated than those of Bagging's base models. Recalling the formula from Section 2.4,

$$\mathrm{Var}(\bar{y}) = \rho\,\sigma^2 + \frac{1-\rho}{T}\,\sigma^2,$$

a higher pairwise correlation ρ means a smaller variance reduction. Stacking therefore inherits a weaker variance-reduction effect than Bagging when the base models belong to the same family.

However, Stacking typically uses *heterogeneous* base learners (e.g., a decision tree, a logistic regression, a neural network, and so on). Models from different families encode different inductive biases and make errors in structurally different regions of the input space. This cross-family diversity can drive ρ substantially lower than within-family diversity, recovering much of the variance benefit even without bootstrap resampling.

Variance in Stacking

Stacking reduces variance when the base learners are sufficiently diverse. Heterogeneous base learner families are the primary mechanism: models with different inductive biases tend to produce less correlated errors than models of the same family trained on different data subsets.

Effect on bias

A single model has a fixed inductive bias determined by its hypothesis class. When multiple base learners with *different* biases produce predictions, the meta-model receives a richer input space that spans the union of those hypothesis classes. If a linear model systematically underestimates in one region and a tree model overestimates there, the meta-model can learn to compensate, effectively reducing the residual bias that any single model would leave.

More precisely, suppose base learner h_k has bias $b_k(\mathbf{x}) = \mathbb{E}[h_k(\mathbf{x})] - f(\mathbf{x})$, where f is the true function. The meta-model h' takes $(h_1(\mathbf{x}), \ldots, h_K(\mathbf{x}))$ as input and learns weights $\boldsymbol{\beta}$ such that

$$H(\mathbf{x}) = h'(h_1(\mathbf{x}), \ldots, h_K(\mathbf{x})) \approx \sum_{k=1}^{K} \beta_k \, h_k(\mathbf{x}).$$

The bias of the ensemble is then

$$\mathbb{E}[H(\mathbf{x})] - f(\mathbf{x}) = \sum_{k=1}^{K} \beta_k \, b_k(\mathbf{x}).$$

If the biases $b_k(\mathbf{x})$ have *opposite signs* in certain regions (that is, some base learners systematically overestimate where others underestimate) the meta-model can assign weights that cause the errors to cancel, yielding a lower residual bias than any individual $b_k(\mathbf{x})$ alone. This cancellation is not guaranteed: it requires that the base learners be diverse enough that their bias functions are not all aligned in the same direction.

Bias in Stacking

Stacking can reduce bias when base learners with complementary bias structures are combined. The meta-model learns to exploit regions where one learner's systematic overestimation cancels another's underestimation. This requires genuine diversity in the inductive biases of the base models, not merely in their random initialisation.

Why the answer is "it depends"

Table 2 summarises the three design choices that jointly determine whether Stacking achieves bias reduction, variance reduction, both, or neither. No single choice guarantees a good outcome in isolation.

Table 2: Design choices and their effect on the bias–variance behaviour of a Stacking ensemble.

Design choice	**Favours variance reduction**	**Favours bias reduction**
Base learner diversity	Heterogeneous families $\Rightarrow$ low ρ	Complementary bias structures $\Rightarrow$ cancellation
Meta-model expressiveness	Simple meta-model (e.g. linear) avoids overfitting	Expressive meta-model can capture complex combination patterns
Validation strategy	Cross-validation prevents optimistic predictions, reducing apparent variance	Out-of-fold predictions expose true base-model errors for the meta-model to correct

The interaction between these choices also introduces a specific risk not present in Bagging or Boosting: if the meta-model is **too expressive** relative to the amount of training data available at Level 2, it may overfit to the particular combination of errors produced by the base models on the training set. This inflates variance at Level 2 and can erase the gains achieved at Level 1. A simple meta-model (logistic regression or a linear combination) is therefore often preferred over a deep neural network or a full decision tree.

Comparison with Bagging and Boosting

The three paradigms now have parallel theoretical characterisations:

	Bagging	**Boosting**	**Stacking**
Variance effect	Guaranteed reduction ($\propto 1/T$ as $\rho \to 0$)	May increase with overfitting	Reduced if base learners are diverse
Bias effect	Unchanged	Guaranteed reduction ($\leq e^{-2\gamma^2 T}$)	Reduced if biases are complementary
Formal guarantee	Yes (Section 2.4)	Yes (Section 3.4)	No (esign-dependent)
Primary condition	Low correlation ρ	Weak learner margin $\gamma > 0$	Base learner diversity + appropriate meta-model

The absence of a formal guarantee for Stacking is not a weakness of the method; it reflects the fact that Stacking is more general and more flexible than the other two paradigms. That flexibility is precisely what makes it powerful, but it also means that the practitioner bears more responsibility for design choices.

A useful heuristic

In practice, a Stacking ensemble tends to improve over its best base learner when at least one of the following holds: (a) the base learners have substantially different error profiles on the validation set; (b) no single base learner dominates all others across all input regions; (c) the meta-model is trained with proper cross-validation and kept simple. If all base learners make similar errors, Stacking degenerates towards a weighted average and provides little benefit over the best individual model.

Entry Ticket 5

A team builds a Stacking ensemble with three base learners: two Random Forests trained with different hyperparameters, and one logistic regression. The meta-model is a deep neural network with five hidden layers.

1. The two Random Forests are from the same family. How does this affect the expected variance reduction of the ensemble, compared with using three models from different families?
2. The deep neural network meta-model achieves near-zero training error on the meta-dataset $\mathcal{D}'$ but poor test performance. What is happening, and how should the team redesign the meta-model?
3. A colleague suggests replacing the logistic regression with a third Random Forest, arguing that "more trees always help". Do you agree? Justify your answer in terms of bias and variance.

4.7 Implementation in Scikit-learn

As shown in Listing 3, the stacking model is implemented using heterogeneous base learners combined through a logistic regression meta-model.

```
...

base_learners = [
    ("logreg", make_pipeline(StandardScaler(),
                             LogisticRegression(max_iter=1000))),
    ("nn",     make_pipeline(StandardScaler(),
                             MLPClassifier(hidden_layer_sizes=(8,),
                                           max_iter=1000))),
    ("dt",     DecisionTreeClassifier(random_state=42)),
    ("knn",    make_pipeline(StandardScaler(),
                             KNeighborsClassifier(n_neighbors=5)))
]

meta_model = LogisticRegression(max_iter=1000)

stack_model = StackingClassifier(
    estimators=base_learners,
    final_estimator=meta_model,
    passthrough=False
)

```

```python
stack_model.fit(X_train, y_train)
y_pred = stack_model.predict(X_test)
print(f"Stacking accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

Listing 3: Stacking classifier with heterogeneous base learners in scikit-learn

Common Mistakes in Stacking

- Training the meta-model on predictions from the same data used to train the base models (data leakage).
- Using a single base model type, which reduces diversity and offers little benefit over a single model.
- Forgetting to retrain base models on the full dataset before deploying the stacking ensemble.

Entry Ticket 6

A team is building a Stacking ensemble for a tabular classification task.

1. Why is it necessary to use cross-validation to generate the training data for the meta-model?
2. The team decides to use a deep neural network as the meta-learner. What risk does this introduce?
3. One team member suggests including the original features of $\mathcal{D}$ together with the base-model predictions in $\mathcal{D}'$. Under what circumstances could this be beneficial?

5 Comparison of Ensemble Methods

5.1 Bias and Variance Considerations

Bagging and Boosting have opposite effects on the bias–variance trade-off:

	Effect on Bias	**Effect on Variance**
Bagging	Unchanged or slightly increased	Strongly reduced
Boosting	Strongly reduced	May increase if overfitted
Stacking	Depends on base and meta models	Can reduce both, if designed carefully

Interpretation. Bagging stabilizes predictions by averaging over independent perturbations of the training data. Boosting improves the model's expressiveness by sequentially correcting mistakes. Stacking integrates multiple perspectives to exploit complementary strengths.

5.2 When to Use Each Method

Method	When to Prefer	Example Use Case
Bagging	High-variance learners; need for stability	Weather prediction with decision trees
Boosting	Weak learners; need for bias correction	Credit scoring, spam detection
Stacking	Heterogeneous models; need for combination	Complex tabular datasets

5.3 Common Pitfalls

- **Overfitting in Boosting**: occurs when too many iterations are used or when base learners become overly complex. Mitigate with early stopping and a small learning rate.
- **Data leakage in Stacking**: happens when base model predictions on the training set are used directly to train the meta-model without cross-validation. Mitigate by using out-of-fold predictions.
- **Computational cost**: ensembles are typically more expensive to train and interpret than single models. Mitigate with parallelization (Bagging) or early stopping (Boosting).

Key Principle

Ensemble learning improves generalization by combining *diverse* models whose errors are not perfectly correlated. Diversity is the essential ingredient: an ensemble of identical models provides no benefit over a single model.

6 Modern Extensions

6.1 Random Forests

Random Forests are one of the most successful practical implementations of Bagging. They extend the bootstrap aggregation idea by adding an additional layer of randomness in the feature space.

Mechanism. Each decision tree is trained on a bootstrap sample, and at each split only a random subset of $\sqrt{p}$ features (out of the total p) is considered. This additional randomization reduces the correlation among trees, which further decreases the ensemble variance beyond what bootstrap sampling alone achieves.

Advantages.

- Robust to overfitting even with deep trees.
- Handles high-dimensional data well.

- Provides built-in feature importance estimates via impurity reduction or OOB error.

Limitation. Interpretability decreases as the forest grows, since no single tree captures the ensemble's decision logic. The model behaves as a black box.

6.2 Gradient Boosting Machines

Gradient Boosting generalizes the original Boosting idea into a gradient-based optimization framework. Instead of focusing only on classification errors, it minimizes a general differentiable loss function through additive model updates.

Concept. The ensemble model is expressed as an additive expansion:

$$F_M(\mathbf{x}) = \sum_{m=1}^{M} \gamma_m \, h_m(\mathbf{x}),$$

where $h_m(\mathbf{x})$ are weak learners (typically shallow trees), and γ_m are step sizes obtained by minimizing the loss. Each new base learner h_m is trained to approximate the **negative gradient** of the loss function with respect to the current model predictions, effectively performing gradient descent in function space.

Modern variants. Popular libraries implementing this approach include XGBoost, LightGBM, and CatBoost. These implementations incorporate regularization, advanced sampling strategies, and parallelization, making them highly efficient for large-scale datasets.

6.3 Deep Ensembles

With the rise of deep learning, ensemble concepts have been extended to neural networks. A *deep ensemble* consists of multiple independently trained neural networks whose outputs are averaged at inference time.

Motivation. Because neural networks are trained with random initialization and stochastic optimization, they can converge to different local minima. Averaging predictions from several such networks tends to improve both accuracy and uncertainty calibration.

Properties.

- Often outperform single large models in predictive accuracy.
- Provide better uncertainty estimates than deterministic networks.
- Can approximate Bayesian model averaging without explicit posterior inference.

Challenge. Training multiple deep networks is computationally expensive, though modern parallel architectures mitigate this limitation.

6.4 Nested Ensembles

Motivation

The ensemble methods discussed so far (Bagging, Boosting, and Stacking) combine *individual* models as their base learners. A natural extension is to allow the base learners themselves to be ensembles. The resulting construction is called a **nested ensemble** (also known as *ensemble of ensembles* or *multi-level ensemble*).

Nested Ensemble

A **nested ensemble** is a hierarchical predictive model in which at least one component is itself an ensemble. The simplest case has two levels: a set of ensemble base learners whose outputs are combined by a second-level aggregator, which may also be an ensemble.

The motivation mirrors the original argument for ensembles: just as combining diverse models reduces error relative to any single model, combining diverse *ensembles* can reduce error relative to any single ensemble, provided the component ensembles make sufficiently uncorrelated mistakes.

Architecture

Figure 5 illustrates the general two-level structure. At Level 1, ensembles $E_1, E_2, \ldots, E_K$ are trained independently; each E_k is itself a collection of base models. At Level 2, a meta-aggregator $\mathcal{A}$ combines the outputs of the Level-1 ensembles. The meta-aggregator can be as simple as a majority vote or as expressive as a learned meta-model (i.e. a Stacking layer).

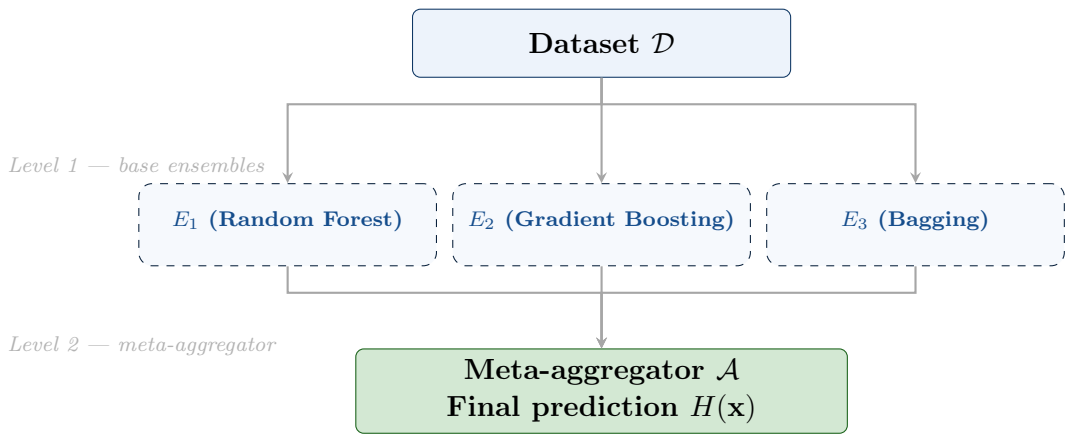

Figure 5: A two-level nested ensemble. Three Level-1 ensembles (E_1: Random Forest, E_2: Gradient Boosting, E_3: Bagging) each aggregate their own base models independently. A Level-2 meta-aggregator $\mathcal{A}$ then combines the outputs of the three ensembles into the final prediction $H(\mathbf{x})$.

Why nesting helps

Variance reduction in a flat ensemble saturates as the number of base models grows, because the irreducible correlation floor $\rho\sigma^2$ (Section 2.4) is bounded by the shared inductive

bias of the base learner family. Nesting introduces a second axis of diversity: the Level-1 ensembles can belong to *different* families (e.g. trees, linear models, neural networks), each with its own bias structure. The Level-2 aggregator then exploits complementarity across families rather than within a single one.

Formally, if the Level-1 ensembles have pairwise output correlation $\rho' < \rho$ (the within-family correlation of the flat ensemble), the variance of the nested prediction is strictly lower than that of any single Level-1 ensemble, by the same argument as in Section 2.4.

Common instantiations

In practice, nested ensembles appear in several recognisable forms:

- **Stacking over ensembles.** The most common pattern: the base learners in a Stacking architecture are themselves Random Forests or Gradient Boosting Machines rather than single decision trees. This is the standard approach in machine learning competitions such as Kaggle, where stacking layers of GBMs routinely yields top-ranked solutions.
- **Cascaded boosting.** Multiple Gradient Boosting pipelines are trained on different feature subsets or data transformations, and their residuals feed a final boosting stage.
- **Deep ensembles of ensembles.** Several independently trained deep ensembles (Section 5.3) are themselves averaged. Used in safety-critical applications where uncertainty calibration is paramount.

Limitations

- **Computational cost.** Training K Level-1 ensembles each containing T base models requires $K \times T$ model fits, which can be prohibitive for large datasets. Parallelisation mitigates training time but not memory cost.
- **Overfitting risk at Level 2.** If the Level-2 meta-aggregator is trained on the same data used to train the Level-1 ensembles, it receives overoptimistic predictions, leading to data leakage (Section 4.3). Cross-validation must be applied at every nesting level, compounding the data-management complexity.
- **Interpretability.** Even a single Random Forest is already difficult to interpret; a nested construction essentially forfeits any direct model explanation. Feature-importance estimates become unreliable and should be replaced by model-agnostic methods (e.g. SHAP values).
- **Diminishing returns.** Empirically, the gain from adding a second nesting level is almost always smaller than the gain from going from a single model to a flat ensemble. A third level rarely yields measurable improvement.

When to use nested ensembles

Nested ensembles are most beneficial when (a) a diverse set of strong ensemble methods is available, (b) computational resources are not the binding constraint, and (c) the task demands the last fraction of predictive accuracy (e.g. competitive

machine learning, high-stakes decision support). In most production settings a single well-tuned Gradient Boosting Machine or Random Forest remains the more practical choice.

Entry Ticket 7

A data scientist is building a nested ensemble for a fraud-detection task. She uses three Level-1 ensembles (Random Forest, XGBoost, and a Bagging of logistic regression models) combined via a logistic regression meta-model.

1. Why is it important that the three Level-1 ensembles belong to different algorithmic families?
2. She trains the Level-1 ensembles on the full training set and then fits the meta-model on their predictions of the same data. What problem does this introduce, and how should it be corrected?
3. Under what circumstances would a single XGBoost model be preferable to this nested construction?

7 Final Remarks

7.1 Choosing the Right Ensemble Method

There is no universal recipe, but the following guidelines apply in most situations:

- Use **Bagging** or **Random Forests** when the base learner is unstable and variance is the dominant source of error.
- Use **Boosting** (AdaBoost, Gradient Boosting) when the base learner is weak and bias is the dominant source of error.
- Use **Stacking** when multiple diverse algorithms are available and you wish to learn their optimal combination from data.

Trade-off. As we move from simpler to more sophisticated methods, both flexibility and computational cost increase:

$$\text{single model} \rightarrow \text{Bagging} \rightarrow \text{Boosting} \rightarrow \text{Stacking} \rightarrow \text{nested ensembles}$$

(increasing flexibility and computational cost)

Each step along this progression yields diminishing marginal returns in predictive accuracy while increasing training cost and reducing interpretability. In most practical settings, a well-tuned Random Forest or Gradient Boosting Machine already captures the bulk of the achievable gain; nested ensembles are justified primarily when the task demands the last fraction of accuracy and resources permit.

7.2 Validation of Ensemble Methods

All ensemble methods must be evaluated using proper validation procedures (see the companion notes on Model Selection). Key considerations include:

- **Bagging**: the out-of-bag (OOB) error provides a "free" cross-validation estimate without a separate validation set.
- **Boosting**: monitor the validation error at each round to avoid overfitting; use early stopping when the validation error begins to increase.
- **Stacking**: always use cross-validation to generate out-of-fold predictions for training the meta-model; never let the meta-model train on predictions from data its base models were trained on.

Key Takeaways

- Ensemble learning combines multiple models to achieve better generalization than any single model.
- Bagging reduces variance; Boosting reduces bias; Stacking combines strengths.
- Diversity among base models is the fundamental requirement for ensemble improvement.
- Proper validation (cross-validation, OOB error) is essential to avoid overly optimistic performance estimates.
- Modern variants such as Random Forests and Gradient Boosting Machines represent the practical state of the art for structured data.

7 References

Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2 edition, 2009.